

SENU — System for Efficiency, Notes, and Utilities

# CostINR	2000
Difficulty	Medium
InternshipRelevance	Strong productivity engineering project demonstrating embedded systems, human-computer interaction, and PC integration; useful secondary portfolio evidence for embedded software and IoT internships.
KeyEvidence	Working prototype demo + GitHub repository + system architecture diagram + ESP32 firmware + Python PC-monitoring service + synchronization and usability test results.
Repo	https://github.com/AlcesOzdst/SENU
Status	In progress
Type	Embedded Systems / Productivity Automation

Project snapshot

> **Working definition:** SENU - System for Efficiency, Notes, and Utilities is a compact desk-side productivity device intended to make frequent actions such as checking tasks, starting a focus session, viewing PC status, and capturing short voice notes accessible through dedicated physical controls. The central question is whether this physical interface provides a measurable advantage over the existing software-only workflow.

Problem: Frequent productivity actions can require switching between applications, reaching for a phone, or breaking focus. SENU explores whether a dedicated physical interface can reduce this interaction friction without becoming another device that needs constant maintenance.

Hypothesis: For a small set of frequent desk actions, dedicated physical controls may reduce completion time, interaction count, and context switching compared with the current laptop-based workflow.

Current status: Planning / pre-prototype. Concept, scope, hardware inventory, preliminary architecture, and evaluation approach are defined. Circuit, GPIO map, firmware, desktop agent, integration, voice recording, enclosure, and measured usability results are not yet verified.

Next milestone: Complete hardware feasibility checks, finalize the ESP32-C3 pin map, and get the standalone OLED + RTC + input UI working.

Research questions

GPIO feasibility

- Can the ESP32-C3 SuperMini accommodate the OLED, DS3231, rotary encoder, 8 tactile buttons, status LED, I²S audio, and microphone input simultaneously?
- Can the design avoid boot/strapping pin conflicts and accidental peripheral contention?
- Is a PCF8574 or MCP23017 GPIO expander actually required?

- What is the simplest reliable pin allocation?

Status: Open.

Audio recording

- Can the bare 2-pin electret microphone be interfaced reliably through a MAX9814 preamplifier?
- What sample rate, bit depth, and mono format are practical on the actual board?
- Can the ESP32-C3 write audio reliably to a microSD card without dropped samples or corrupted files?
- Does the result provide enough quality for short voice notes?
- Does voice recording justify its added hardware/software complexity?

Status: Open / feasibility experiment required.

Power compatibility

- Is the available 12 V / 5 V supply regulated?
- Which modules can safely receive 5 V and which require 3.3 V?
- Is current capacity sufficient for the ESP32-C3, OLED, audio amplifier, speaker, storage, and peripherals?
- What wiring/protection is required to prevent accidental overvoltage?

Status: Open / must be verified before full wiring.

Local-network security

- How should the device authenticate with the local FastAPI service?
- What data is exposed over HTTP?
- What happens if an untrusted device joins the same LAN?
- Should SENU use a shared secret, device token, mTLS, or an isolated local-network assumption for the first prototype?
- What information must never leave the local network?

Status: Open.

Physical controls vs software shortcuts

- Are physical controls actually faster than keyboard shortcuts, desktop widgets, or existing applications?
- For which actions does a dedicated device provide enough frequency/value to justify its hardware?
- Does SENU reduce context switching or simply move the interaction to another screen?
- What is the maintenance overhead relative to the time saved?

Status: Core research question.

Sources & findings

Use this section as a running research database.

Source / link	Date accessed	Topic	Key finding	Reliability	Design implication
Espressif ESP32-C3 documentation		MCU / GPIO		High / Medium / Low	

Source / link	Date accessed	Topic	Key finding	Reliability	Design implication
PlatformIO ESP32 documentation		Firmware		High / Medium / Low	
Adafruit SSD1306 documentation		OLED		High / Medium / Low	
DS3231 documentation / module reference		RTC		High / Medium / Low	
MAX9814 documentation		Microphone		High / Medium / Low	
MAX98357 documentation		Audio output		High / Medium / Low	

Research note template

Question:

Why it matters:

Source(s):

Finding:

Fact / assumption / inference:

Impact on SENU design:

Decision / next experiment:

Date reviewed:

Architecture & design decisions

Proposed architecture



Core responsibilities

ESP32-C3

- Local UI and physical input handling
- Clock / RTC display
- Offline focus timer
- Device state and connectivity status
- Audio capture/playback control

Python desktop agent

- Local API
- Task/deadline data
- Windows CPU/RAM metrics
- SQLite persistence
- Device synchronization

Design principle: Keep the core experience functional when Wi-Fi is unavailable. Network-dependent data should show a clear disconnected or stale state and recover gracefully after reconnection.

Decision log

Decision	Alternatives considered	Reason	Trade-offs	Date
Use 128×64 SSD1306 OLED	Larger TFT	Already owned; lower cost and simpler power/UI requirements	Less screen area and less graphical richness	04/10/2026
Use PlatformIO + Arduino framework	Arduino IDE / ESP-IDF	Practical development workflow while retaining ESP32 flexibility	Less low-level control than pure ESP-IDF	
Use HTTP/JSON initially	WebSockets	Simpler architecture for request/response actions	Not ideal for continuous real-time updates	
Use SQLite locally	Cloud database	Offline-first and low operational overhead	Single-device/local scope	
Defer final enclosure until core functions are validated	Build enclosure early	Avoid polishing an architecture that may change	Prototype remains less polished initially	

Hardware & BOM

Owned hardware

Component	Ownership	Price	Supplier	Voltage	Interface	Status
ESP32-C3 SuperMini	Owned			3.3 V logic	GPIO / I²C / I²S / ADC	Available
0.96-inch SSD1306 OLED	Owned			Verify module	I²C	Available
Rotary encoder	Owned			Verify	GPIO	Available
8 tactile buttons	Owned			Verify	GPIO	Available
DS3231 RTC	Owned			Verify module	I²C	Available
MAX98357	Owned			Verify board	I²S	Available
0.5 W speaker	Owned			Verify impedance	Audio	Available
2-pin electret microphone	Owned			Verify bias	Analog	Needs preamp
Single-colour LED	Owned			Verify resistor	GPIO	Available
Breadboard / wires / headers / USB	Owned			n/a	n/a	Available
12 V / 5 V power supply	Owned			Verify output	Power	Must test

Tentative purchases

Component	Why	Planning estimate	Supplier	Status
microSD SPI module + card	Local voice-note storage	₹150–₹350		To buy

Component	Why	Planning estimate	Supplier	Status
Enclosure materials	Mechanical protection / finish	₹100–₹300		To buy later
Resistor / capacitor kit	Only for missing passive components	₹30–₹100		Check inventory
PCF8574 or MCP23017	May expand GPIO capacity	₹50–₹180		Conditional

Estimated remaining hardware cost: ₹430–₹1,180 excluding shipping and any power-supply replacement. This is a planning estimate, not a live quote.

Hardware checks before wiring

- Confirm the supply is regulated and current-rated.
- Never connect 12 V directly to 3.3 V or 5 V modules.
- Verify MAX98357 supply limits and speaker impedance.
- Verify microphone polarity and MAX9814 wiring.
- Confirm actual I²C addresses; the OLED is commonly 0x3C / 0x3D, while DS3231 is commonly 0x68.
- Create and test a full ESP32-C3 pin map before connecting all peripherals.

Experiments & test results

Experiment database

Test ID	Hypothesis	Setup	Procedure	Measurements	Outcome	Next action
EXP-001	OLED and DS3231 can coexist on I²C	ESP32-C3 + OLED + RTC	Scan bus, read RTC, render time	Addresses / failures		
EXP-002	All required controls fit on available GPIO	Final candidate pin map	Assign and test every peripheral	GPIO conflicts / boot issues		
EXP-003	FastAPI agent returns reliable PC metrics	Windows PC + local LAN	Repeated API calls	Latency / error rate		
EXP-004	Voice audio can be recorded reliably to microSD	Mic + MAX9814 + microSD	Repeated fixed-length recordings	Write failures / audio quality / file integrity		
EXP-005	Physical UI can outperform software workflow for selected tasks	Baseline laptop workflow vs SENU	Alternating repeated trials	Time / inputs / switches / errors / effort		

Usability evaluation

Compare SENU with the current laptop-only workflow using the same tasks, multiple trials, and alternating method order to reduce learning effects.

Candidate tasks

1. Check the next task/deadline.
2. Start a focus session.
3. Check CPU/RAM status.

4. Capture a short note if voice recording is implemented.
5. Recover from a disconnected Wi-Fi state.

Record

- Completion time in seconds
- Physical inputs or clicks
- Application / context switches
- Errors or failed interactions
- Subjective effort on a consistent scale
- Sync / reconnect failures
- Setup and maintenance overhead

Calculations

Time reduction (%) = (Baseline − SENU) / Baseline × 100

Input reduction (%) = (Baseline inputs − SENU inputs) / Baseline inputs × 100

Net weekly time saved = (seconds saved per use × uses per week) / 60

Simple break-even time = total build cost / value of time saved per unit of time

Do not claim productivity improvement unless the measurements support it.

Build log

Date	Milestone	Change made	Files / commit	Result	Issue encountered	Next action
04/10/2026	Project setup	Renamed project to SENU and defined research log	Notion / GitHub	Documentation structure established	Hardware not yet validated	Finalize pin map

Risks & blockers

Risk	Likelihood	Impact	Mitigation	Owner	Status
ESP32-C3 GPIO exhaustion	Medium	High	Make and test pin map; add expander only if required	Parth	Open
Audio recording proves unreliable	Medium	High	Prototype recording before enclosure work	Parth	Open
Power-supply mismatch	Medium	High	Measure output and verify module voltage/current requirements	Parth	Open
Wi-Fi / API failure harms usability	Medium	Medium	Offline core functions + explicit stale/disconnected state	Parth	Open
Physical device offers no meaningful benefit	Medium	High	Run controlled baseline comparison	Parth	Open

Risk	Likelihood	Impact	Mitigation	Owner	Status
Scope creep from too many features	High	Medium	Keep a strict MVP and defer non-critical features	Parth	Open

Weekly review

Week of

Progress:

What was learned:

What failed:

Evidence collected:

Biggest blocker:

Decision made:

Next week’s priority:

Weekly questions

- What changed technically?
- What assumption was disproved?
- What evidence do I have?
- What should I stop building?
- What is the single most important next experiment?

Portfolio evidence

Track only evidence that actually exists.

Evidence	Status	Link / file	What it proves
System architecture diagram	Planned		Shows component and data flow
Hardware wiring / pin map	Planned		Shows implementation choices
Working prototype photos	Planned		Shows physical build
Firmware repository	Planned	https://github.com/AlcesOzdst/SENU	Shows embedded implementation
Desktop agent	Planned		Shows PC integration
API / database documentation	Planned		Shows system design
Usability test results	Planned		Supports or rejects efficiency hypothesis
Demo video	Planned		Shows end-to-end functionality

Portfolio positioning: SENU should be presented as an evidence-driven embedded productivity project combining embedded systems, human-computer interaction, and local Windows integration. The strongest portfolio claim will be a reliable prototype with measured usability and reliability results rather than a long feature list.

Suggested Notion databases

Create these as linked databases or child databases as the project grows.

Research Sources

Properties: Title, URL, Date accessed, Topic, Summary, Reliability, Design impact.

Build Log

Properties: Date, Milestone, Change, Result, Git commit, Blocker, Next action.

Experiments

Properties: Test ID, Hypothesis, Method, Baseline, SENU result, Conclusion, Evidence.

Tasks

Properties: Task, Priority, Status, Milestone, Due date, Blocked by.

Recommended status values

Tasks: Not started → In progress → Blocked → Testing → Done

Experiments: Planned → Running → Passed → Failed → Inconclusive

Research: To review → Reviewed → Decision made

Engineering principles for SENU

1. Evidence over assumptions

Do not label planned features as implemented.

2. Reliability over feature count

A smaller reliable device is more valuable than a feature-rich unstable device.

3. Offline-first for core actions

Clock and focus timer should not depend on Wi-Fi.

4. Measure before claiming efficiency

Physical controls are only better when measured against the existing workflow.

5. Delay irreversible work

Do not finalize the enclosure or buy unnecessary hardware before feasibility testing.

6. Document why, not only what

Every important hardware or software choice should have a recorded reason and trade-off.

SENU — Production Board

SENU — Production Board

Task	Priority	Milestone	Due Date	Blocked By
Run baseline vs SENU usability experiment	P0 — Critical	M6 — Evaluation & Portfolio		
Validate power supply output and current capacity	P0 — Critical	M0 — Feasibility		
Create complete ESP32-C3 GPIO and peripheral pin map	P0 — Critical	M0 — Feasibility		

Task	Priority	Milestone	Due Date	Blocked By
Run repeated voice recording integrity test	P0 — Critical	M4 — Voice POC		
Add local-network authentication strategy	P0 — Critical	M3 — Integration		
Implement disconnected / stale-data UI state	P1 — High	M3 — Integration		
Implement OLED menu and navigation state machine	P1 — High	M1 — Standalone UI		
Document reliability and measured results	P1 — High	M6 — Evaluation & Portfolio		
Verify OLED I²C address and standalone display	P1 — High	M0 — Feasibility		
Implement MAX98357 playback	P1 — High	M4 — Voice POC		
Finalize README, diagrams, BOM and demo	P1 — High	M6 — Evaluation & Portfolio		
Implement WAV/PCM recording to microSD	P1 — High	M4 — Voice POC		
Test reconnect and API failure behavior	P1 — High	M3 — Integration		
Create FastAPI desktop-agent scaffold	P1 — High	M2 — Desktop Agent		
Implement task/deadline API endpoints	P1 — High	M2 — Desktop Agent		
Verify DS3231 I²C address and RTC read/write	P1 — High	M0 — Feasibility		
Implement 8-button input matrix / GPIO strategy	P1 — High	M1 — Standalone UI		
Add psutil CPU/RAM metrics endpoint	P1 — High	M2 — Desktop Agent		
Buy / validate microSD SPI module and card	P1 — High	M4 — Voice POC		
Design SQLite schema for tasks and focus sessions	P1 — High	M2 — Desktop Agent		
Prototype microphone + MAX9814 ADC input	P1 — High	M4 — Voice POC		
Implement offline Pomodoro/focus timer	P1 — High	M1 — Standalone UI		
Integrate ESP32 with local HTTP/JSON API	P1 — High	M3 — Integration		
Implement rotary encoder input handling	P1 — High	M1 — Standalone UI		
Prototype OLED + DS3231 together on I²C	P1 — High	M1 — Standalone UI		
Add status LED states	P2 — Medium	M1 — Standalone UI		
Design and build final enclosure	P2 — Medium	M5 — Enclosure		